PRZYKŁAD	ZADANIE
Baza danych: szpital	Baza danych: hotel

INF.03 · Tworzenie i administrowanie bazami danych

Relacje w bazach danych — XAMPP / phpMyAdmin / MySQL

8 zadań · każde zadanie zawiera gotowy przykład na bazie szpital i analogiczne polecenie do samodzielnego wykonania na bazie hotel

Z1 Rodzaje relacji: 1:1, 1:N, M:N

PRZYKŁAD 🏥 Szpital — rozpoznaj typ relacji

Przeanalizuj poniższe schematy i określ typ relacji.

LEKARZ	→	PACJENT
PK id_lekarza	1 : N	FK id_lekarza
imie, nazwisko		imie, nazwisko

Jeden lekarz prowadzący → wielu pacjentów. Relacja: 1:N

LEKARZ	↔	PACJENT
PK id_lekarza	M : N	PK id_pacjenta
	tab. łącząca: WIZYTY	

Lekarz przyjmuje wielu pacjentów, pacjent odwiedza wielu lekarzy. Relacja: M:N

LEKARZ	→	GABINET
PK id_lekarza	1 : 1	FK id_lekarza (UNIQUE)

Każdy lekarz ma dokładnie jeden gabinet. Relacja: 1:1

ZADANIE 🏨 Hotel — rozpoznaj typ relacji

Dla każdej pary tabel wpisz typ relacji (1:1 / 1:N / M:N) i uzasadnij w jednym zdaniu.

✏ *POKOJ ↔ REZERWACJA — typ relacji i uzasadnienie:*

✏ *GOSC ↔ POKOJ (jeden gość może wynajmować wiele pokoi, pokój może być zajęty przez różnych gości) — typ relacji:*

✏ *PRACOWNIK ↔ ETAT (każdy pracownik ma dokładnie jeden etat) — typ relacji:*

Z2 Klucz główny (PK) i klucz obcy (FK) — relacja 1:N

PRZYKŁAD 🏥 Szpital — tabele oddzialy i lekarze

```
CREATE DATABASE szpital CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE szpital;

CREATE TABLE oddzialy (
    id_oddzialu  INT AUTO_INCREMENT PRIMARY KEY,
    nazwa        VARCHAR(100) NOT NULL,
    pietro       INT
) ENGINE=InnoDB;

CREATE TABLE lekarze (
    id_lekarza    INT AUTO_INCREMENT PRIMARY KEY,
    imie          VARCHAR(80) NOT NULL,
    nazwisko      VARCHAR(80) NOT NULL,
    specjalizacja VARCHAR(100),
    id_oddzialu   INT,
    FOREIGN KEY (id_oddzialu)
        REFERENCES oddzialy(id_oddzialu)
        ON DELETE SET NULL
        ON UPDATE CASCADE
) ENGINE=InnoDB;

INSERT INTO oddzialy (nazwa, pietro) VALUES
    ('Kardiologia', 2), ('Chirurgia', 3), ('Pediatria', 1);

INSERT INTO lekarze (imie, nazwisko, specjalizacja, id_oddzialu) VALUES
    ('Anna', 'Kowalska', 'kardiolog', 1),
    ('Piotr', 'Nowak', 'chirurg', 2),
    ('Marta', 'Wisniewska', 'pediatra', 3);
```

Relacje działają TYLKO z silnikiem InnoDB! Sprawdź: tabela → Operacje → Silnik przechowywania.

ZADANIE 🏨 Hotel — tabele typy_pokoi i pokoje

Utwórz bazę hotel i dwie tabele połączone relacją 1:N:

- Tabela typy_pokoi — kolumny: id_typu, nazwa, cena_za_noc, opis
- Tabela pokoje — kolumny: id_pokoju, numer, pietro + klucz obcy do typy_pokoi
- Dodaj 3 typy pokoi i 5 pokoi jako dane testowe

✏ *Kod SQL — CREATE TABLE typy_pokoi:*

✏ *Kod SQL — CREATE TABLE pokoje (z FOREIGN KEY):*

✏ *Jakie ON DELETE wybrałeś/aś i dlaczego?*

Z3 Relacja M:N — tabela łącząca

PRZYKŁAD 🏥 Szpital — tabele pacjenci i wizyty

```
USE szpital;

CREATE TABLE pacjenci (
    id_pacjenta  INT AUTO_INCREMENT PRIMARY KEY,
    imie         VARCHAR(80),
    nazwisko     VARCHAR(80),
    pesel        CHAR(11) UNIQUE
) ENGINE=InnoDB;

CREATE TABLE wizyty (
    id_wizyty    INT AUTO_INCREMENT PRIMARY KEY,
    id_pacjenta  INT NOT NULL,
    id_lekarza   INT NOT NULL,
    data_wizyty  DATE NOT NULL,
    diagnoza     TEXT,
    FOREIGN KEY (id_pacjenta)
        REFERENCES pacjenci(id_pacjenta) ON DELETE CASCADE,
    FOREIGN KEY (id_lekarza)
        REFERENCES lekarze(id_lekarza) ON DELETE CASCADE
) ENGINE=InnoDB;

INSERT INTO pacjenci (imie, nazwisko, pesel) VALUES
    ('Jan', 'Kowalski', '80010112345'),
    ('Ewa', 'Nowak', '90050567890');

INSERT INTO wizyty (id_pacjenta, id_lekarza, data_wizyty, diagnoza) VALUES
    (1, 1, '2024-03-01', 'Nadcisnienie'),
    (1, 2, '2024-03-05', 'Konsultacja'),
    (2, 1, '2024-03-10', 'Badanie kontrolne');
```

Tabela wizyty to tabela łącząca — zawiera dwa klucze obce (id_pacjenta i id_lekarza) tworząc relację M:N.

ZADANIE 🏨 Hotel — tabele goscie i rezerwacje

Rozszerz bazę hotel o tabelę goscie i tabelę łączącą rezerwacje:

- Tabela goscie — kolumny: id_goscia, imie, nazwisko, email, telefon
- Tabela rezerwacje — kolumny: id_rezerwacji, FK→goscie, FK→pokoje, data_od, data_do, cena_calkowita
- Dodaj 2 gości i 3 rezerwacje

✏ *Kod SQL — CREATE TABLE goscie:*

✏ *Kod SQL — CREATE TABLE rezerwacje (z dwoma FOREIGN KEY):*

✏ *Kod SQL — INSERT INTO goscie i rezerwacje (dane testowe):*

Z4 INNER JOIN — łączenie tabel

PRZYKŁAD 🏥 Szpital — lekarze z oddziałem + lista wizyt

```
-- Lekarze z nazwą oddziału
SELECT l.imie, l.nazwisko, l.specjalizacja, o.nazwa AS oddzial
FROM lekarze l
INNER JOIN oddzialy o ON l.id_oddzialu = o.id_oddzialu
ORDER BY o.nazwa;

-- Wizyty: pacjent + lekarz + data (3 tabele)
SELECT
    p.imie AS pacjent, p.nazwisko,
    l.nazwisko AS lekarz,
    w.data_wizyty, w.diagnoza
FROM wizyty w
INNER JOIN pacjenci p ON w.id_pacjenta = p.id_pacjenta
INNER JOIN lekarze  l ON w.id_lekarza  = l.id_lekarza
ORDER BY w.data_wizyty DESC;
```

INNER JOIN zwraca tylko wiersze mające dopasowanie w obu tabelach. Lekarz bez oddziału (NULL) nie pojawi się w wynikach.

ZADANIE 🏨 Hotel — pokoje z typem + rezerwacje z gościem

- Zapytanie 1: Wszystkie pokoje z nazwą i ceną swojego typu (JOIN pokoje + typy_pokoi)
- Zapytanie 2: Rezerwacje z imieniem i nazwiskiem gościa oraz numerem pokoju (JOIN trzech tabel)

✏ *Zapytanie 1 — pokoje z typem pokoju:*

✏ *Zapytanie 2 — rezerwacje z gościem i numerem pokoju:*

Z5 LEFT JOIN — rekordy bez powiązań

PRZYKŁAD 🏥 Szpital — lekarze bez wizyt

```
-- Dodaj lekarza bez zadnych wizyt
INSERT INTO lekarze (imie, nazwisko, specjalizacja, id_oddzialu)
VALUES ('Adam', 'Stary', 'dermatolog', 1);

-- Wszyscy lekarze z liczbą wizyt (nawet 0)
SELECT l.imie, l.nazwisko,
       COUNT(w.id_wizyty) AS liczba_wizyt
FROM lekarze l
LEFT JOIN wizyty w ON l.id_lekarza = w.id_lekarza
GROUP BY l.id_lekarza, l.imie, l.nazwisko
ORDER BY liczba_wizyt DESC;

-- Tylko lekarze BEZ zadnych wizyt
SELECT l.imie, l.nazwisko
FROM lekarze l
LEFT JOIN wizyty w ON l.id_lekarza = w.id_lekarza
WHERE w.id_wizyty IS NULL;
```

LEFT JOIN zwraca WSZYSTKIE wiersze z lewej tabeli, nawet jeśli nie mają dopasowania po prawej stronie (wtedy kolumny prawej tabeli = NULL).

ZADANIE 🏨 Hotel — pokoje bez rezerwacji

- Dodaj do tabeli pokoje co najmniej jeden pokój bez żadnej rezerwacji
- Zapytanie 1: Wszystkie pokoje z liczbą rezerwacji (LEFT JOIN + COUNT, wyniki też dla 0)
- Zapytanie 2: Tylko pokoje, które nie mają żadnej rezerwacji (WHERE ... IS NULL)

✏ *Zapytanie 1 — pokoje z liczbą rezerwacji:*

✏ *Zapytanie 2 — tylko wolne pokoje:*

✏ *Czym różni się wynik INNER JOIN od LEFT JOIN na tych danych?*

Z6 Integralność referencyjna — ON DELETE

PRZYKŁAD 🏥 Szpital — testy usuwania rekordów

```
-- TEST 1: Usun oddział z lekarzami (ON DELETE SET NULL)
DELETE FROM oddzialy WHERE id_oddzialu = 1;
-- Efekt: lekarze z tego oddzialu maja teraz id_oddzialu = NULL
SELECT id_lekarza, imie, nazwisko, id_oddzialu FROM lekarze;

-- TEST 2: Usun pacjenta z wizytami (ON DELETE CASCADE)
DELETE FROM pacjenci WHERE id_pacjenta = 1;
-- Efekt: jego wizyty rowniez zostaly usuniete
SELECT COUNT(*) AS pozostale_wizyty FROM wizyty;
```

Opcja	Co się dzieje z rekordami-dziećmi?	Kiedy stosować?
SET NULL	FK ustawiany na NULL	Relacja opcjonalna (np. lekarz bez oddziału)
CASCADE	Dzieci usuwane automatycznie	Dzieci nie mają sensu bez rodzica (wizyty)
RESTRICT	Błąd — usunięcie zablokowane	Gdy integralność jest kluczowa

ZADANIE 🏨 Hotel — testy usuwania w bazie hotel

- TEST 1: Spróbuj usunąć typ pokoju, który ma przypisane pokoje. Zapisz wynik i wyjaśnij.
- TEST 2: Usuń gościa z rezerwacjami. Sprawdź SELECT COUNT(*) FROM rezerwacje przed i po usunięciu.

✏ *TEST 1 — wynik i wyjaśnienie:*

✏ *TEST 2 — liczba rezerwacji przed i po usunięciu gościa:*

✏ *Czy ON DELETE które zastosowałeś/aś w hotelu było odpowiednie? Uzasadnij.*

Z7 GROUP BY i HAVING — agregacja danych z relacji

PRZYKŁAD 🏥 Szpital — statystyki lekarzy i oddziałów

```
-- Ilu pacjentow (unikalnych) przyjal kazdy lekarz?
SELECT l.imie, l.nazwisko,
       COUNT(DISTINCT w.id_pacjenta) AS liczba_pacjentow
FROM lekarze l
LEFT JOIN wizyty w ON l.id_lekarza = w.id_lekarza
GROUP BY l.id_lekarza, l.imie, l.nazwisko
HAVING COUNT(w.id_wizyty) >= 1
ORDER BY liczba_pacjentow DESC;

-- Ile lekarzy ma kazdy oddzial?
SELECT o.nazwa AS oddzial,
       COUNT(l.id_lekarza) AS liczba_lekarzy
FROM oddzialy o
LEFT JOIN lekarze l ON o.id_oddzialu = l.id_oddzialu
GROUP BY o.id_oddzialu, o.nazwa
ORDER BY liczba_lekarzy DESC;
```

WHERE filtruje wiersze PRZED grupowaniem. HAVING filtruje grupy PO agregacji (po GROUP BY). Użyj HAVING gdy warunek dotyczy COUNT, SUM, AVG itp.

ZADANIE 🏨 Hotel — statystyki rezerwacji

- Zapytanie 1: Ile rezerwacji ma każdy gość? (GROUP BY gość, ORDER BY malejąco)
- Zapytanie 2: Które typy pokoi są najczęściej rezerwowane? (COUNT + GROUP BY + ORDER BY)
- Zapytanie 3: Pokaż tylko typy pokoi z więcej niż 1 rezerwacją (HAVING)

✏ *Zapytanie 1 — rezerwacje per gość:*

✏ *Zapytanie 2 — popularność typów pokoi:*

✏ *Zapytanie 3 — typy pokoi z HAVING COUNT > 1:*

Z8 Projekt końcowy — nowa tabela i relacja

PRZYKŁAD 🏥 Szpital — tabele leki i recepty (nowa relacja M:N)

```
USE szpital;

CREATE TABLE leki (
    id_leku  INT AUTO_INCREMENT PRIMARY KEY,
    nazwa    VARCHAR(150) NOT NULL,
    dawka    VARCHAR(50)
) ENGINE=InnoDB;

CREATE TABLE recepty (
    id_recepty        INT AUTO_INCREMENT PRIMARY KEY,
    id_pacjenta       INT NOT NULL,
    id_leku           INT NOT NULL,
    id_lekarza        INT NOT NULL,
    data_wystawienia  DATE,
    FOREIGN KEY (id_pacjenta) REFERENCES pacjenci(id_pacjenta) ON DELETE CASCADE,
    FOREIGN KEY (id_leku)     REFERENCES leki(id_leku)         ON DELETE CASCADE,
    FOREIGN KEY (id_lekarza)  REFERENCES lekarze(id_lekarza)   ON DELETE CASCADE
) ENGINE=InnoDB;

-- Recepty z danymi pacjenta, lekarza i leku
SELECT p.imie AS pacjent, p.nazwisko,
       l.nazwisko AS lekarz,
       lk.nazwa AS lek, r.data_wystawienia
FROM recepty r
JOIN pacjenci p  ON r.id_pacjenta = p.id_pacjenta
JOIN lekarze  l  ON r.id_lekarza  = l.id_lekarza
JOIN leki     lk ON r.id_leku     = lk.id_leku
ORDER BY r.data_wystawienia DESC;
```

ZADANIE 🏨 Hotel — zaprojektuj i dodaj nową tabelę

Rozszerz bazę hotel o jedną nową tabelę powiązaną relacją z istniejącymi tabelami. Wybierz jeden pomysł:

- uslugi (spa, parking, restauracja) + zamowienia_uslug (tabela łącząca z rezerwacjami)
- opinie — oceny wystawiane przez gości po pobycie (FK→goscie, FK→pokoje)
- pracownicy (recepcjoniści) + przypisanie pracownika do rezerwacji

✏ *Jaką tabelę dodajesz i jaki typ relacji z którymi tabelami?*

✏ *Kod SQL — CREATE TABLE nowej tabeli:*

✏ *Kod SQL — INSERT INTO (minimum 3 rekordy):*

✏ *Zapytanie SELECT z JOIN pokazujące dane z nowej relacji:*

Tabela wyników

Nr	Zadanie	Przykład ✓	Zadanie ✓
1	Rodzaje relacji: 1:1, 1:N, M:N		
2	Klucz główny (PK) i klucz obcy (FK)		
3	Relacja M:N — tabela łącząca		
4	INNER JOIN — łączenie tabel		
5	LEFT JOIN — rekordy bez powiązań		
6	Integralność referencyjna — ON DELETE		
7	GROUP BY i HAVING		
8	Projekt — nowa tabela i relacja		

Imię i nazwisko: _____

Klasa: _____ Data: _____ Ocena: _____